

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO4: Examine intermediate code generation techniques to enhance code optimization (BL4)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:50

Annexure A

INDUSTRY PROBLEM- SOLVING TASK

Code of Conduct:

I P.Karpaga shree(192424042) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

P.Karpaga shree(192424042)

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 25	/ 20	/ 20	/ 10	/ 100

Annexure A

INDUSTRY PROBLEM- SOLVING TASK

1. Banking Transaction Processing System

A banking software company is developing a compiler for a domain-specific scripting language used in transaction processing systems. The compiler must translate variable declarations, assignment statements, and Boolean expressions efficiently to avoid transaction failures during peak hours. The system also uses case statements to classify transactions such as deposit, withdrawal, and loan processing. During testing, incorrect intermediate code generation caused delays in transaction execution.

Tasks:

- (i) Design suitable intermediate code for the given assignment and Boolean expressions. (K3)
- (ii) Explain how backpatching can help in handling conditional transaction validation. (K4)
- (iii) Analyze the role of intermediate languages in improving banking software portability. (K5)

2. Smart Traffic Control System

An automotive company is building a smart traffic management application where signals are controlled automatically based on traffic density. The compiler used in the embedded controller must process Boolean expressions and case statements for different traffic conditions. Due to inefficient intermediate code generation, the response time of traffic signals increased significantly during rush hours. The development team plans to optimize procedure calls and assignment statements in the compiler design.

Tasks:

- (i) Construct intermediate code for the traffic signal control logic. (K3)
- (ii) Explain how procedure calls improve modularity in the traffic control application. (K4)
- (iii) Evaluate the importance of compiler optimization in real-time embedded systems. (K5)

3. E-Commerce Product Recommendation Engine

An e-commerce company uses a rule-based recommendation engine developed using a custom programming language. The compiler processes declarations, assignment statements, and Boolean conditions

to generate recommendations dynamically. During seasonal sales, delays occurred because of improper handling of case statements and inefficient intermediate code generation. The organization wants to redesign the compiler to improve execution speed and maintainability.

Tasks:

- (i) Generate intermediate representation for the recommendation logic. (K3)
- (ii) Discuss how case statements can efficiently handle product category selection. (K4)
- (iii) Analyze the role of compiler design in improving large-scale e-commerce applications. (K5)

4. Hospital Patient Monitoring System

A healthcare technology company is developing a patient monitoring system that continuously checks vital signs using embedded software. The compiler used in the system must efficiently evaluate Boolean expressions and assignment statements for emergency alerts. Improper backpatching in conditional branching caused delayed alarm generation during critical patient conditions. The software team is redesigning the compiler for faster and more reliable intermediate code execution.

Tasks:

- (i) Design intermediate code for patient condition monitoring logic. (K3)
- (ii) Explain the significance of backpatching in emergency alert systems. (K4)
- (iii) Evaluate how compiler optimization improves reliability in healthcare applications. (K5)

5. Online Airline Reservation System

An airline company is modernizing its reservation platform using a custom scripting language for ticket booking and seat allocation. The compiler must handle declarations, assignment statements, and procedure calls efficiently to manage thousands of booking requests simultaneously. During testing, errors in intermediate code generation affected seat allocation and payment confirmation processes. The software architects aim to improve compiler performance for large-scale distributed applications.

Tasks:

- (i) Develop intermediate code for ticket booking and seat allocation operations. (K3)
- (ii) Explain how procedure calls support modular reservation management. (K4)
- (iii) Analyze the role of intermediate languages in distributed airline reservation systems.

SET 1-ANSWER

Banking Transaction Processing System

Problem Statement

A banking software company is developing a compiler for a domain-specific scripting language used in transaction processing systems. The compiler must translate variable declarations, assignment statements, and Boolean expressions efficiently to avoid transaction failures during peak hours. The system also uses case statements to classify transactions such as deposit, withdrawal, and loan processing. During testing, incorrect intermediate code generation caused delays in transaction execution.

Tasks:

- (i) Design suitable intermediate code for the given assignment and Boolean expressions. (K3)
- (ii) Explain how backpatching can help in handling conditional transaction validation. (K4)
- (iii) Analyze the role of intermediate languages in improving banking software portability. (K5)
-

(i) Intermediate Code for the Assignment and Boolean Expressions [K3]

1. Problem Understanding

Consider the core banking rule that must be enforced before a withdrawal transaction is committed: a withdrawal is permitted only when the account balance is sufficient AND the account status is active. This involves two categories of statements that the compiler must translate:

- Assignment statements — e.g., `balance = balance - amount`, `transactionFlag = 1`
- A Boolean expression combining a relational and a logical operator — e.g., `(balance >= amount)` and `(status == 1)` — used inside an if-then-else construct that drives transaction validation.

The scenario is modeled using the following representative source fragment, which captures declarations, assignment, and Boolean-controlled branching exactly as described in the problem:

```

if (balance >= amount) and (status == 1) then
    balance = balance - amount
    transactionFlag = 1
else
    transactionFlag = 0

```

2. Method Selection

Three-Address Code (TAC) in quadruple form is selected as the intermediate representation because it is close to machine code yet machine-independent, making it ideal for a transaction engine that must later be retargeted to different banking hardware/VMs. For the Boolean expression, the classical jumping-code (backpatching) technique — rather than numerical (0/1) representation — is chosen, because:

- Transaction validation is used purely for control flow (branching), not for storing a Boolean value, so jumping code avoids the overhead of computing and storing an intermediate truth value.
- It naturally supports short-circuit evaluation of the AND operator, which is critical when peak-hour transaction volume makes every avoided evaluation valuable.
- It allows single-pass code generation using synthesized attributes truelist and falselist together with backpatch(), avoiding a second pass to fix up jump targets — essential for compiling transaction scripts fast enough to keep up with peak-hour load.

Grammar used (Dragon-book style translation scheme for Boolean expressions and if-then-else statements):

```

E -> E1 and M E2 { backpatch(E1.truelist, M.instr);
                    E.truelist = E2.truelist;
                    E.falselist = merge(E1.falselist, E2.falselist) }
E -> id1 relop id2 { E.truelist = makelist(nextinstr);
                    E.falselist = makelist(nextinstr+1);
                    emit('if' id1.place relop.op id2.place 'goto _'); emit('goto _') }
M -> ε { M.instr = nextinstr } S -> if
    E then M1 S1 N else M2 S2
{ backpatch(E.truelist, M1.instr);

```

```

        backpatch(E.falselist, M2.instr);
        S.nextlist = merge(S1.nextlist, N.nextlist, S2.nextlist) }
N -> ε      { N.nextlist = maelist(nextinstr); emit('goto _') }

```

3. Solution Process (Step-by-Step Derivation)

Step 1 — Translate the relational sub-expression E1: $\text{balance} \geq \text{amount}$

```

100: if balance >= amount goto _    ( E1.truelist = {100} )
101: goto _    ( E1.falselist = {101} )

```

Step 2 — Marker M records the start of E2 ($\text{nextinstr} = 102$), then translate E2: $\text{status} == 1$

```

102: if status == 1 goto _          ( E2.truelist =
{102} ) 103: goto _    ( E2.falselist = {103} )

```

Step 3 — Combine E1 and M E2 for the AND operator

```

backpatch(E1.truelist, M.instr = 102) -> instr 100 becomes:
100: if balance >= amount goto 102

E.truelist = E2.truelist = {102}
E.falselist = merge(E1.falselist, E2.falselist) = {101, 103}

```

Step 4 — Marker M1 records start of the then-branch ($\text{nextinstr} = 104$); backpatch E.truelist to 104

```

instr 102 becomes:
102: if status == 1 goto 104

```

Step 5 — Generate code for the then-branch S1 (the two assignment statements)

```

104: t1 = balance - amount    ; TAC for balance = balance - amount
105: balance = t1
106: transactionFlag = 1

```

Step 6 — N generates the unconditional jump that skips the else-branch once nextinstr is known ($=107$), and records it in nextlist

```

107: goto _    ( N.nextlist = {107} )

```

Step 7 — Marker M2 records start of the else-branch (nextinstr = 108); backpatch E.falselist to 108

```
instr 101 becomes: 101: goto 108
instr 103 becomes: 103: goto 108
```

Step 8 — Generate code for the else-branch S2

```
108: transactionFlag = 0
```

Step 9 — After the whole if-statement, backpatch S.nextlist ({107}) to the next available instruction (109)

```
instr 107 becomes: 107: goto 109
109: ... ( next statement in the program )
```

4. Accuracy of Calculation — Final Backpatched Intermediate Code

Addr	Quadruple / Instruction
100	if balance >= amount goto 102
101	goto 108
102	if status == 1 goto 104
103	goto 108
104	t1 = balance - amount
105	balance = t1
106	transactionFlag = 1
107	goto 109
108	transactionFlag = 0
109	(next statement)

Verification by tracing both outcomes confirms correctness of the generated code:

- Case $\text{balance} \geq \text{amount}$ AND $\text{status} == 1$ (valid withdrawal): 100 falls through to 102, 102 falls through to 104-106 → balance and transactionFlag are updated correctly, then 107 jumps to 109, skipping instruction 108. ✓
- Case $\text{balance} < \text{amount}$ (insufficient funds): 100 jumps to 108 directly → transactionFlag = 0, then falls through to 109 without touching balance. ✓
- Case $\text{balance} \geq \text{amount}$ but $\text{status} \neq 1$ (inactive account): 100 falls through to 102, 102 jumps to 108 → transactionFlag = 0, balance unchanged. ✓

All three control paths reach the correct target address with no dangling or missing jump target — confirming that every entry in truelist/falselist was backpatched exactly once and the quadruple sequence is fully resolved.

(ii) Role of Backpatching in Conditional Transaction Validation [K4]

Backpatching is the technique of generating branch instructions with temporarily unfilled (blank) jump targets, tracked using truelist/falselist attributes, and filling in the actual target addresses once they become known later in the same pass, using the backpatch() procedure.

Its role in this banking scenario is explained below:

- Single-pass, peak-hour-safe compilation: Since jump targets are patched in the same pass instead of requiring a separate fix-up pass over the syntax tree, the compiler can generate validation code for deposit, withdrawal, and loan transactions quickly enough to keep up with high transaction volumes during peak hours.
- Natural support for short-circuit logic: For $(\text{balance} \geq \text{amount})$ and $(\text{status} == 1)$, backpatching ensures that if the balance check fails, control jumps straight to the failure branch (108) without ever evaluating the status check — avoiding unnecessary computation on every declined transaction.
- Clean handling of nested/chained validations: Real banking rules often chain multiple conditions (fraud check, daily-limit check, KYC status). Backpatching scales naturally — each additional 'and'/'or' term simply extends the truelist/falselist merge chain without restructuring already-generated code.
- Correct control-flow linking for if-then-else: The marker nonterminal M records the instruction number where a branch destination begins before that code exists; N records the jump that must skip the else-branch. Backpatching is what allows these forward references to be resolved safely and exactly once.

- Reliability during runtime: Because every truelist/falselist entry is guaranteed to be backpatched, there are no unresolved or incorrect jump targets — directly preventing the kind of intermediate-code generation defects that caused transaction execution delays described in the problem statement.

(iii) **Role of Intermediate Languages in Improving Banking Software Portability [K5]**

Intermediate representations such as three-address code decouple the front end (lexical analysis, parsing, and semantic checking of the banking scripting language) from the back end (code generation for a specific target machine, VM, or runtime). This separation directly benefits a banking software company in the following ways:

- Machine independence: TAC/quadruples do not assume any particular register set or instruction format, so the same intermediate code produced from a transaction script can be translated to run on mainframes, cloud microservice runtimes, or ATM embedded controllers by writing only a new back-end code generator.
- Retargetability with a single front end: When the bank migrates transaction processing from on-premise servers to a distributed cloud platform, only the back-end target-code generator needs to change — the front end that understands the banking DSL's declarations, assignments, and Boolean/case constructs remains untouched, drastically cutting redevelopment cost and risk.
- Target-independent optimization: Passes such as common sub-expression elimination, constant folding, and dead-code elimination can be performed once on the intermediate code (e.g., simplifying the $t1 = \text{balance} - \text{amount}$ computation) and automatically benefit every deployment target, instead of re-implementing optimizations for each platform.
- Consistency and auditability across deployments: Because every target is generated from the same verified intermediate representation, the bank can be confident that transaction-validation logic (balance and status checks) behaves identically on the head office server, the disaster-recovery site, and mobile-banking backends — an important regulatory and reliability requirement for financial software.
- Faster, safer maintenance: Bug fixes or new validation rules (e.g., adding a fraud-check condition) are implemented once at the intermediate-code level and propagate correctly to every target platform the back end supports, reducing the class of errors — like the incorrect intermediate code generation mentioned in the case — that previously caused transaction delays.

In summary, the intermediate language acts as a portable, optimizable, and verifiable 'contract' between the banking DSL and the many hardware/software platforms a modern banking system must run on, which is precisely what allows the compiler redesign effort in this case to improve both reliability and portability simultaneously.

RUBRICS FOR CALCULATION

Numerical Problems					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

